

HOW SERGE WORKS

Logic & decision trees of the Serge coding agent — the free-tier hive with one personality. Snapshot: 2026-07-11. Runs on Linux, macOS (launchd), and Windows (WSL2 or Git Bash).

1 What Serge is

Serge is a terminal AI coding agent — a fork of *openclaude* (a Claude-Code-compatible engine) that runs entirely on **cloud free tiers** (Mistral, Google, Cerebras), wired together by a local **LiteLLM router** on `localhost:4000`. It behaves like a small **hive** of specialists: different models take different jobs, but every seat is injected with the same **constitution** (`~/.serge/CONSTITUTION.trimmed.v3.md`) — one personality, many brains. Around the engine sits a shell of **hooks**: session-start memory loaders, an end-of-turn verify → review → gate pipeline, a spend watchdog, and autonomous loops that run Serge headlessly on timers.

THE GOLDEN RULE: plentiful seats by default; the scarce deep seat only when it earns it. The cheapest brain that CAN do the job — and nothing ships unchecked.

2 The hive — seats & roster

Seat name	Model (2026-07)	~Free quota	Role
local-coder	Mistral Large	~1B tok/month	Default workhorse — ~90% of turns
fast-coder	Devstral Medium	(same Mistral pool)	The scout — codebase recon, burst fan-outs
think-coder	Magistral Medium	(same Mistral pool)	Deep reasoner — adversarial passes, /deepthink
qwen-coder	gpt-oss-120b (Cerebras)	~1M tok/day	The reviewer — independent second opinion, different vendor on purpose
pro-coder	Gemini 3.1 Flash-Lite	own daily bucket	Classifier / live-fact single-shot calls (kept off the driver's quota)
cloud-brain / bedrock-brain	Gemini 3.5 Flash	~20 req/day	The architect — hardest reasoning + the only VISION seat
free-flash / -flash3 / -flash25	Gemini buckets	~20/day each	Fallback rungs — per-model quota, so each hop is fresh budget
free-qwen	Qwen3 Coder 480B (OpenRouter :free)	~50 req/day	4th-provider rung before anything paid
opus-paid / haiku-paid / kimi-coder	Claude Opus 4.6 / Haiku 4.5 / Kimi K2.5 (Bedrock)	PAID	Escape hatches — bill only when named; opus is in NO fallback chain

Seats are defined in `~/.serge/litellm.yaml`; the router hides them all behind one OpenAI-compatible endpoint, so the engine just asks for a model name.

3 Decision tree — launching `serge`

```
$ serge [--cloud|--council|--terse|--auto|--yolo|--uncap]
|
|-- source ~/.serge/serge.env          (search keys, routing knobs, budgets)
|-- parse mode flags
|   |-- council -> append council.md system prompt, SERGE_REVIEW_ENFORCE=1
|   |-- terse   -> append lean.md output-discipline overlay
|   |-- auto    -> permission-mode acceptEdits   (auto-apply edits only)
|   |-- yolo    -> bypassPermissions + ENFORCE review hooks as compensation
|
|-- daily-spend cap check (~/.serge/monitor/.budget-capped)
|   capped? --uncap given -> clear flag, re-baseline today's spend, go
|   else      -> explain the pause + how to resume, EXIT
|
|-- router health: GET localhost:4000/v1/models
|   down? -> start it, wait <=45s (LiteLLM cold boot ~30s):
|       linux: systemctl --user start serge-router
|       no systemd (macOS / Git Bash / bare WSL): spawn litellm
|       directly with router.env loaded -> monitor/router.log
|   still down? -> print fix commands, EXIT
|
|-- exec node dist/cli.mjs with OPENAI_MODEL = local-coder   (default)
|                           or bedrock-brain (--cloud)
|                           [OPENAI_MODEL=<seat> overrides per session]
```

4 Decision tree — which brain answers this turn?

Re-evaluated on **every human turn** inside the engine's OpenAI shim (`openaiShim.ts`); injected `<system-reminder>` text is stripped before classifying so hidden context can't trigger escalation. Escalations hold for the whole tool loop, then flip back on the next easy turn.

```
You send a prompt
|
v
[1] session started with --cloud? .....> bedrock-brain (Gemini 3.5)
|
[2] message contains an IMAGE? .....> SERGE_VISION_MODEL
|   (only the brain seat has vision)      (= bedrock-brain)
|
[3] live-fact question? (scores, prices, weather,
|   news – cheap coders hallucinate current facts) ..> SERGE_LIVEFACT_MODEL
|                                                         (= cloud-brain)
|
[4] "hard task" classifier regex hits?
|   architecture / redesign / trade-offs / race
|   condition / deadlock / memory leak / root-cause /
|   security-critical / crypto / distributed txn ...
|   PLUS practical-debug signals: "still not working",
|   "error when", "keeps crashing", stack trace,
|   regression, build failure .....> SERGE_CLASSIFIER_MODEL
|                                                         (= cloud-brain)
|
[5] otherwise – the usual ~90% .....> local-coder (Mistral) [DEFAULT]

Reasoning depth (orthogonal): /effort low..xhigh
- seats in SERGE_THINKING_SEATS (opus/haiku-paid only) get native Anthropic
  thinking budgets (xhigh = 16384 tokens)
- every other seat gets reasoning_effort (xhigh clamps to high);
  Gemini/gpt-oss/Magistral honor it, Mistral Large ignores it (non-reasoner)
```

5 Fallback chains — when a seat 429s or dies mid-stream

Fallbacks fire **only on error** (throttle / 5xx / mid-stream drop). Chains hop **providers** — one account's outage or quota can never collapse a whole chain. Gemini free quota is per-model, so each Gemini bucket is a fresh ~20/day. Only the workhorse chain ends on a paid rung (Serge must not brick mid-task); everything else stays free and fails loudly.

```
local-coder (Mistral Large)                                THE WORKHORSE CHAIN
-> free-flash25 (Gemini 2.5 Flash) -> free-flash3 (Gemini 3 Flash)
-> free-flash (3.1 Flash-Lite)    -> free-qwen (OpenRouter :free)
-> haiku-paid (Bedrock, PAID — the only paid rung anywhere)

cloud-brain / bedrock-brain (Gemini 3.5)                  THE BRAIN CHAIN
-> free-flash3 -> free-flash25 -> free-flash -> local-coder
(Gemini buckets first — they drive agentic loops fine; NEVER Cerebras,
whose models hang Claude-Code-style tool loops — eval-verified)

qwen-coder (reviewer) -> pro-coder -> free-flash          opus-paid -> (nothing:
fast-coder (scout)    -> free-flash -> free-scout         fails loudly rather
think-coder (reasoner)-> free-brain -> pro-coder          than bill silently)
```

6 Decision tree — end of turn (the Stop pipeline)

Two Stop hooks run in order (`settings.json`). Exit code 2 from a hook **blocks the stop** and feeds its message back to Serge — that's how the hive forces itself to fix things.

```
Serge tries to end its turn
|
v
[A] continue-on-unfinished.sh          THE ANTI-STALL CHECK
| transcript claims ongoing work ("let me check...") but no tool ran,
| or claims "done" with zero tools used?
| YES -> exit 2: "finish the job" (anti-loop cap; disable:
|     SERGE_AUTOCONTINUE_DISABLE)      -> Serge keeps working
| no -> continue
|
v
[B] stop-checks.sh — one process, three stages:
|
| STAGE 1: VERIFY (free, deterministic — verify-on-stop.sh)
| typecheck + lint + hygiene greps (debug lines, merge markers,
| .only tests) on files edited this turn
| FAIL + enforce -> exit 2, Serge fixes, re-runs ONCE
|                 (repeat failures land in the reflexion log)
| hard fail      -> SKIP the review (code is about to change)
|
| STAGE 2: REVIEW (free AI — review-on-stop.sh)
| qwen-coder (gpt-oss, DIFFERENT VENDOR) reads the turn's diff
| BLOCKING bug + enforce (--council/--yolo) -> brain seat writes a
|     fix plan -> exit 2 -> Serge applies it, ONCE
| nits / clean -> advisory note only
|
| STAGE 3: GATE (only if Serge's OWN brain files changed — gate-on-stop.sh)
| files in SERGE_GATE_HASH_GLOBS (constitution, council, agents) hashed;
| changed? -> run the golden-task eval suite vs baseline
| regression -> exit 2: revert-or-justify (the self-edit ratchet;
|     a PreToolUse hook also gates constitution WRITES)
|
v
[C] turn ends -> reflexion-log.sh appends lessons; statusline shows session cost
```

7 Session start — what Serge remembers

Loader (SessionStart hook)	Injects
memory-load.sh	The memory INDEX (memory/MEMORY.md) — one line per durable fact; full fact files opened only when relevant. Add facts with /remember .
progress-load.sh	Where it left off on ongoing tasks (also after /clear & compaction).
reflexion-load.sh	Past REPEATED verify failures — real tool output, so it stops tripping on the same stone.
plan-pointer.sh	Pointer to a previously approved plan (plan.md) — plan-high / execute-cheap.
notifications-load.sh	Pending items from the autonomous loops (NOTIFICATIONS.md).
stale-binary-check.sh	Warns when the running build is older than the source (a /clear keeps old code!).

8 Autonomous loops — Serge without prompts

Outer loops (~/ .serge/loops/) prompt Serge **headlessly** on triggers; the human feeds queues and reviews results. Every loop follows the same shape:

```
trigger (timer / path-watch) -> guards (kill switches, daily caps, locks)
-> build_prompt() = the SENSOR (deterministic; often decides "nothing to do" for $0)
-> serge -p --yolo = the ACTUATOR (bounded: --max-turns, budget, timeout(1))
-> tripwire          (did Serge touch its OWN brain files? -> ALL loops hard-disable)
-> journal (one JSON line/run) -> post_run() (branches, inboxes, notifications)
```

Loop	Trigger	Sensor -> action
eval-gate	nightly 03:30	Run the golden-task gate; on regression: diagnose, write findings, propose (never apply) constitution fixes.
err-triage	serge-query-errors.log grows	Classify new errors vs known causes → triage-inbox.md ; new signatures get a draft memory note for human promotion.
backlog	hourly (ships disabled)	Drain one - [] item from an enrolled project's BACKLOG.md in an isolated worktree on branch loop/<slug> ; tests gate the ✓.

Rails: global kill switch touch ~/ .serge/loops/DISABLED (per-loop: <name>/DISABLED); per-loop + global daily run caps; every run bounded in turns, dollars and wall-clock; loops may only ever **propose** changes to the brain — the tripwire + eval ratchet enforce it.

9 Memory & self-improvement

```
+-----+
task completed ---->| reflexion-log.sh: what failed, what |----+
verify failed ---->| worked (real tool output, not vibes) |  |
+-----+
/remember --> ~/.serge/memory/.md (index: MEMORY.md, loaded each session)
                                |
bigger lesson? ----> constitution-proposals.md (HUMAN reviews & applies)
                        |
                        v
gate-on-constitution-edit.sh (PreToolUse) + nightly eval-gate loop:
any brain edit must NOT regress the 10 golden tasks vs baseline
(01 smoke ... 05 no-hardcoded-secret ... 07 honesty>approval ... 10 no-fabricated-data)
```

Net effect: Serge learns durable facts freely, but its **personality can only be changed through a regression-gated, human-approved path** — even by itself.

10 Cost discipline

Mechanism	What it does
All-free roster	Every working seat is a \$0 tier; session cost line normally reads ≈ \$0.00.
Budget watchdog (5-min timer)	Pauses the router when paid (OpenRouter) daily spend crosses the cap (drop-in: \$10/day). <code>serge --uncap</code> or UTC midnight resumes.
Paid-seat tripwire	Any Bedrock escape-hatch use is flagged in <code>NOTIFICATIONS.md</code> — a paid seat can never fire silently.
Quota gauge	<code>quota.sh</code> feeds the statusline: today's brain-seat usage vs the ~20/day bucket.
Free-tier scanner (weekly)	Probes providers for new/removed free models; proposes roster upgrades.
Thinking tripwire (daily)	Probes that reasoning depth still actually fires on key seats (providers silently change APIs).

11 Quick reference

Modes & flags

```
serge          the free workhorse
serge --cloud   start on the brain seat
serge --council architect plans, reviewer attacks
serge --terse   lean answers (style only)
serge --auto    auto-apply edits
serge --yolo    hands-off; review hooks enforced
serge --uncap   clear the daily-spend pause
OPENAI_MODEL=<seat> serge  pin any seat
/effort xhigh   max reasoning depth
/cost /recap /hive /remember /deepthink
```

Where things live

```
~/.serge/CONSTITUTION.trimmed.v3.md the personality
~/.serge/litellm.yaml                seats + fallbacks
~/.serge/settings.json               hook wiring
~/.serge/stop-checks.sh              verify->review->gate
~/.serge/loops/                      autonomous loops
~/.serge/memory/                     what Serge remembers
~/.serge/router.env serge.env         THE ONLY KEY FILES
~/programs/serge-0.1.0/               engine (fork) + wrapper
```

In one line: a free workhorse does the work, a second vendor checks it for free, the scarce deep brain is summoned only for the genuinely hard 10% — and the whole hive runs on \$0.

Generated 2026-07-11 from the live install (roster of 2026-07-10). Source: `docs/how-serge-works.html` — edit and re-render with the command in the file header. Companion docs: `serge-0.1.0/SERGE.md` (full reference), `serge-0.1.0/HOW_SERGE_WORKS.txt` (plain-text map), `dot-serge/loops/README.md` (loops).